《算法分析与设计》
课后作业

作业编号_____作业 4_____

学　　号_____

姓　　名_____

专　　业_____

学　　院_____

二 0 二四年四月

第 4 章 分治法

题目 1：设 A[1]=”*”,A[2]为如下的形状：

```
    *
  *   *
    *
```

也就是 A[2]为：

```
      A[1]
  A[1]   A[1]
      A[1]
```

对应的 A[n]为：

```
      A[n-1]
  A[n-1   A[n-1]
      A[n-1]
```

输入要求： 输入整数 n，占 1 行。

输出要求：对应的图形。

样例输入：

3

样例输出：

```
        *
      *   *
        *
    *       *
  *   *   *   *
    *       *
      *   *
        *
```

参考答案：

(1) 设计思路

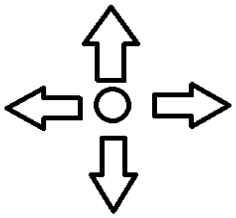
由于字符模式下无法直接指定程序打印字符的坐标， 因此定义一个大的二维数组（由输入 n 确定数组规模），模拟屏幕区域。利用二维数组进行递归，在合适的位置将数组内容置为“*”。最后直接打印出二维数组的内容。

首先确定初始的中心坐标，满足如下关系式：

$$x(n) = 3 * x(n-1)-1 \qquad y(n) = 3 * y(n-1)-1$$

$$且x(1) = 1, y(1) = 1$$

以起始点为中心，向上、下、左、右递归即可，递归的出口是 n = 1 时对应的 x y，直接赋值为 * 。



(2)程序代码

```
#include <vector>
#include <iostream>
using namespace std;
const int maxn = 100;
vector<vector<char>> grid(maxn, vector<char>(maxn, ' '));
void paint(int level, int x, int y, int len){
    if(level == 1){//递归出口
        grid[x][y] = '*';
        return;
    }
    const int d = (2 * len - 1) / 3;//确定两个中心之间距离
    const int nx[] = {x, x, x + d, x - d};//确定下次的 x 坐标
    const int ny[] = {y + d, y - d, y, y};//确定下次的 y 坐标
    for(int i = 0; i < 4; i++){
        paint(level - 1, nx[i], ny[i], len - d);//上下左右递归
    }
}
int main(){
    int n;
    cin >> n;
    int x = 1, y = 1;
    for(int i = 2; i <= n; i++){//确定中心
        x = 3 * x - 1;
        y = 3 * y - 1;
    }
    paint(n, x, y, x);//画图
    for(int i = 1; i < maxn; i++){//输出结果
        for(int j = 1; j < maxn; j++){
            cout << grid[i][j];
        }
```

```
        cout << "\n";
    }
    return 0;
}
```

(3)算法复杂度分析

写出递推方程:

$$T(n) = 4 * T(n-1) \quad ,n > 1$$
$$T(1) = 1 \quad ,n = 1$$

解得$T(n) = 4 \wedge n$

所以复杂度是$O(4^n)$

题目 2：某供电公司在某省各地修建了很多的变电站，这些变电站在出现故障时需要及时的维修，因此派遣距离这些变电站最近的工人去维修将会极大的减少维修的费用。

输入：输入第一行包含一个整数 N ($1 \le N \le 100000$)，表示变电站和工人的数量。其后的 2N 行，每一行有两个整数 X ($0 \le X \le 1000000000$)，Y ($0 \le Y \le 1000000000$)，前面的 N 行表示变电站的位置坐标，后面的 N 行表示工人的位置坐标。

输出：1 行，也就是与这些变电站距离最近的工人所对应的最小距离，精确到小数点后 3 位。

样例输入：

```
4
0 0
0 1
1 0
1 1
2 2
2 3
3 2
3 3
```

样例输出：

```
1.414
```

参考答案：

(1) 设计思路

该问题实质就是最近点对的问题，只是点有两种类型，只能在不同类型的点之间计算距离，同类点之间不计算（即如果两个点类型相同那么返回的距离为无穷大；如果两个点的类型不同，正常计算其距离）。

（2）实现代码

```
#include <vector>
#include <iostream>
#include <tuple>
#include <cmath>
#include <algorithm>
using namespace std;
vector<tuple<long long, long long, long long>> a;
long long inf = 0x3f3f3f3f3f3f3f3f;
inline long long dist(tuple<long long, long long, long long> a, tuple<long long, long long, long long> b){
    //计算路径长度,相同类型的点距离返回无穷大,否则正常计算
    auto[x1, y1, t1] = a;
    auto[x2, y2, t2] = b;
    return t1 == t2 ? inf : (x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2);
}
long long f(int left, int right){
    if(left + 1 == right){//递归出口
        return dist(a[left], a[right]);
    }
    if(left == right) return inf;
    int mid = (left + right) / 2;//确定第几个是中点
    long long temp = min(f(left, mid), f(mid + 1, right));//这里是计算左右两区间最短距离
    //下面是计算两个点在不同的区间的情况
    auto[x, y, t] = a[mid];
    for(int i = mid; i >= left; i--){
        auto [ix, iy, it] = a[i];
        if(ix < x - temp) break;
        for(int j = mid + 1; j <= right; j++){
            auto[jx, jy, jt] = a[j];
            if(jx > x + temp) break;
            temp = min(temp, dist(a[i], a[j]));
        }
    }
    return temp;
}
int main(){
    int n;
    cin >> n;
    for(int i = 0; i < n; i++){
        long long x, y;
        cin >> x >> y;
        a.push_back(make_tuple(x, y, 0));//0 类型点
```

```
    }
    for(int i = 0; i < n; i++){
        long long x, y;
        cin >> x >> y;
        a.push_back(make_tuple(x, y, 1));//1 类型点
    }
    sort(a.begin(), a.end());//排序
    long long ans = f(0, 2 * n - 1);//计算
    //cout << sqrt(ans) << endl;
    printf("%.3lf\n", sqrt(ans));
    return 0;
}
```

(3) 复杂度分析

这道题实际上就是最近点对的稍微变型，复杂度和分治法求最近点对的复杂度一样。

最近点对的递推式：

$$T(n) = 1 \quad ,n <= 3$$
$$T(n) = 2 * T(n/2) + n, n > 3$$

解得的复杂度是$o(n * logn)$

参考资料：

https://blog.csdn.net/heyongluoyao8/article/details/7109739#t0

问题：找出二个集合S1与S2之间的最接近点对的距离（包括一个点集中最接近点对的讨论）复杂度为O（nlogn）

```
5
11 45
14 19
19 810
19 26
8 17
21 23
70 13
1 37
95 7
1 1
3.606
```